ASSIGNMENT 1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Course Outcome: CO1 – Apply OOP principles and C++ constructs

Student Name: D. Gnana Sai

Register Number: 192525121

Department / Section: AIML

Course : DSA0110

Submission Date: 28/10/2026

GitHub Repository: https://github.com/Gnanasai0730/DSA0110-CPP-192525121

Important execution note

This report is a working draft based on the supplied assignment PDF. Replace the student and GitHub placeholders. The included C++ source files were compiled and executed in the available environment where g++ was detected; see each task folder's output.txt. Your assignment requires you to reproduce the runs on your own computer, capture genuine screenshots, and make progressive commits. Do not claim a screenshot, GitHub push, or Google Classroom submission unless you personally complete it.

Compiler command used: g++ -std=c++17 -Wall -Wextra main.cpp -o program.exe

Money representation: all production fare calculations use integer paise. Rs 10 = 1000 paise; Rs 50 = 5000 paise.

Contents

1. Task 1: Understand the Problem and Identify the Objects
2. Task 2: Use Tokens, Keywords, Identifiers and Constants Correctly
3. Task 3: Choose Data Types and Check Type Compatibility
4. Task 4: Control Implicit Conversions
5. Task 5: Write Correct Expressions with the Right Precedence
6. Task 6: Design the Control Structures
7. Task 7: From Procedural to Object-Oriented
8. Task 8: Read and Print Data Safely
9. Task 9: Implement and Test the Fare Logic
10. Task 10: Compare Expectations with Execution
11. Task 11: Recommend and Defend the Final Design
12. Task 12: Re-evaluate and Adapt the Design

TASK 1 – Understand the Problem and Identify the Objects (4 marks)

The system charges Rs 10 for Inner, Rs 20 for Middle and Rs 30 for Outer zone. During peak periods (08:00–09:59 and 17:00–19:59), a 20% surcharge applies. A journey is allowed only if at least Rs 50 remains after deduction. The brief says students and senior citizens receive a concession but does not specify its percentage; this report treats the concession rate as a configurable rule to be clarified rather than inventing a rate.

Objects, data and behaviour

- SmartCard: stores a private balance in paise; can check eligibility, deduct a fare and report the remaining balance. It owns the balance invariant.
- Passenger: stores passenger category (regular, student or senior citizen) and can identify the applicable concession. Separate category data avoids scattered special-case checks.
- Journey: stores origin/destination or farthest zone, hour, date/day type and computed fare; validates journey inputs.
- Zone: represents Inner, Middle, Outer (and later Airport). An enum class prevents accidental mixing with unrelated integers.
- FareTable: stores the base fare for each zone and centralises fare revisions so that the same fare is not typed in multiple functions.

Procedural vs object-oriented

A procedural solution would keep balance, zone, hour and fare in variables and call free functions such as calculateFare() and canTravel(). An OOP solution groups state and behaviour into SmartCard, Journey, Passenger and FareTable classes. OOP is easier to maintain as the system gains new zones, passenger types and passes.

OOP principles

- Encapsulation: SmartCard keeps balance private and permits changes only through validated methods.
- Abstraction: callers use calculateFare() and deduct() without needing to know every arithmetic detail.
- Inheritance: StudentPassenger and SeniorPassenger could derive from Passenger if they share a common interface and different concession policies.
- Polymorphism: a virtual concessionRate() or calculateConcession() method could select the passenger-specific rule at run time.

Journey invariants and assumptions

- Balance after a successful journey must be at least 5000 paise (Rs 50).
- A fare must be non-negative; an invalid zone/hour must be rejected.
- Deduction occurs only after all validation succeeds; failed journeys leave the balance unchanged.
- Peak intervals are half-open: 08:00 is peak, 10:00 is not; 17:00 is peak, 20:00 is not.
- The concession rate and order of applying concession vs surcharge must be specified by the authority before production use.

Alternative concept and complexity
With global variables and free functions, a fare-rule change may require editing multiple functions and makes accidental balance changes easier. Both styles calculate one fare in O(1) time and use O(1) extra space per card. The main difference is maintainability and enforcement of invariants, not asymptotic complexity.

Task 1 demonstration source: Task01/main.cpp. It checks eligibility and deducts through a SmartCard method.

```
#include <iostream>
#include <string>
using namespace std;
class SmartCard {
   long long balancePaise;
public:
   explicit SmartCard(long long initial) : balancePaise(initial) {}
   long long balance() const { return balancePaise; }
   bool canTravel(long long farePaise) const {
      return farePaise >= 0 && balancePaise - farePaise >= 5000;
   }
   bool deduct(long long farePaise) {
      if (!canTravel(farePaise)) return false;
      balancePaise -= farePaise;
      return true;
   }
};
int main() {
   SmartCard card(10000);
   cout << boolalpha << card.canTravel(3000) << '\n';
   card.deduct(3000);
   cout << card.balance() << " paise\n";
}
```

Observed output:

```
true
7000 paise
```

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly (3 marks)

C++ tokens include keywords (class, return, switch), identifiers (SmartCard, balancePaise), constants/literals (5000, 20, true), operators (+=, >=, /) and punctuators ({ } ; , ( )). Identifiers should be meaningful, begin with a letter or underscore, contain no spaces and avoid reserved keywords.

Prefer constexpr for compile-time values such as BASE_PAISE, PEAK_SURCHARGE_PERCENT and MIN_BALANCE_PAISE. const prevents modification after initialisation but is not always a compile-time constant. #define is a preprocessor text substitution and has no type or normal scope; it can cause unexpected substitutions. enum class Zone { Inner, Middle, Outer } is safer than a plain enum because values are scoped and do not implicitly convert to int.

Example constants: constexpr long long MIN_BALANCE_PAISE = 5000; constexpr int PEAK_SURCHARGE_PERCENT = 20; constexpr long long BASE_PAISE[] = {1000, 2000, 3000};

Alternative and complexity: macros generally occupy no separate storage, and constexpr values often require no runtime storage either. Both have O(1) access and do not materially change running time. constexpr and enum class improve type checking and scope; macros do not.

Task 2 demonstration source: Task02/main.cpp.

```
#include <iostream>
using namespace std;
enum class Zone { Inner, Middle, Outer };
constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
int main() {
   cout << BASE_PAISE[0] << ' ' << BASE_PAISE[1] << ' '
        << BASE_PAISE[2] << '\n';
   cout << MIN_BALANCE_PAISE << ' ' << PEAK_SURCHARGE_PERCENT << '\n';
}
```

Observed output:

```
1000 2000 3000
5000 20
```

TASK 3 – Choose Data Types and Check Type Compatibility (4 marks)

Recommended data types

Value	Type	Reason	Typical sizeof
Fare / balance in paise	long long	Exact integer arithmetic; supports large totals	Usually 8 bytes
Hour / zone index	int or enum class	Small whole-number range; enum class improves safety	int usually 4 bytes
Peak / pass active	bool	Two logical states	sizeof(bool) commonly 1 byte
Journey count	long long	Supports millions and future growth	Usually 8 bytes

Display-only ratio	double	Useful for non-currency calculations; not authoritative money storage	Usually 8 bytes
Character	char	Single character input/display	Usually 1 byte
Inferred local	auto	Use when the initialiser makes the type clear	Depends on deduced type

Exact sizes are implementation-dependent; the included program prints sizes on the compiler used. Use <climits> values such as INT_MAX and LLONG_MAX to inspect limits on the target platform.

Money should be stored as integer paise: 10 rupees is 1000 paise. Floating-point values such as 0.1 cannot generally be represented exactly in binary, so repeated additions may introduce small rounding differences. Integer paise avoids this for fares that are whole paise.

Network-scale example: at Rs 30 per journey, 3,000,000 journeys collect Rs 90,000,000 (90 million rupees), equal to 9,000,000,000 paise. This exceeds a signed 32-bit int's typical maximum of 2,147,483,647 paise, so a long long is required for a paise total.

Alternative and complexity: double and long long additions are both O(1), and both commonly use 8 bytes. Double represents a broad range but may lose exact decimal precision; long long is exact for integer paise within its range.

Task 3 demonstration source: Task03/main.cpp.

```
#include <climits>
#include <iostream>
using namespace std;
int main() {
    cout << "sizeof(int)=" << sizeof(int) << '\n';
    cout << "sizeof(long long)=" << sizeof(long long) << '\n';
    cout << "INT_MAX=" << INT_MAX << '\n';
    cout << "LLONG_MAX=" << LLONG_MAX << '\n';
    double rupees = 0.1 + 0.2;
    long long paise = 10 + 20;
    cout.precision(17);
    cout << "double 0.1+0.2=" << rupees << '\n';
    cout << "integer paise total=" << paise << '\n';
}
```

Observed output:

```
sizeof(int)=4
sizeof(long long)=8
INT_MAX=2147483647
LLONG_MAX=9223372036854775807
```

```
double 0.1+0.2=0.30000000000000004
integer paise total=30
```

TASK 4 – Control Implicit Conversions (4 marks)

Prediction written before execution: 3 3.5; C 68; 1 true; s >= u. The fourth line is the surprising one: comparing signed -1 with unsigned 1 converts the signed operand to unsigned, so -1 becomes a large unsigned value and the comparison is false.

Rules: a / b performs integer division when both operands are int, so 7/2 is 3 before assignment to double. static_cast<double>(a)/b makes the division floating-point. 'A' + 2 promotes the character to int for arithmetic; assigning the result to char stores 'C'. Printing ch+1 promotes char to int, producing 68. Assigning 5 to bool yields true; default bool output is 1 and std::boolalpha prints true. Signed/unsigned comparison can trigger -Wsign-compare warnings in similar expressions.

Use static_cast where a conversion is intentional. Brace initialisation such as int x{3.7}; is ill-formed because it would narrow; this is caught at compile time. Explicit casts do not usually change runtime complexity or storage, but make intent visible.

```
#include <iostream>
int main() {
    int a = 7, b = 2;
    double c = a / b;
    double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';
    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';
    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';
    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Actual output from execution:

```
3 3.5
C 68
1 true
s >= u
```

Comparison: the predicted and actual outputs should match on a typical conforming implementation using the default stream formatting. The char numeric code depends on the execution character set; on the target environment it is ASCII-compatible.

TASK 5 – Write Correct Expressions with the Right Precedence (4 marks)

Peak check: (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). The parentheses make the two half-open intervals clear. Surcharge: if (peak) fare += fare * PEAK_SURCHARGE_PERCENT / 100;. Concession should use an explicit, documented policy, for example fare = fare * (100 - concessionPercent) / 100; only after the authority defines the percentage and rounding rule.

Prediction written before execution:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Explanation: multiplication binds more tightly than addition, giving 5+3*2 = 11; parentheses make (5+3)*2 = 16. In x++ + ++y, postfix uses old x=5 then increments x to 6; prefix increments y from 3 to 4 then yields 4; z=9. The conditional expression returns max(6,4)=6; % and * have equal precedence and associate left-to-right, so 6%4*2=4. Integer division truncates toward zero: 10/3=3 and -10/3=-3; the remainder has the dividend's sign here, giving -10%3=-1. The loop adds i=0,3,6,9, giving 18.

Avoid i = i++ because it is confusing and has historically caused undefined/unsequenced behaviour concerns depending on the exact expression/context; use i++ or ++i as a separate statement. For simple standalone increments, their resulting value is irrelevant. For negative operands, C++ integer division truncates toward zero.

```
#include <iostream>
int main() {
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';
    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';
    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

Actual output from execution:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Alternative: bool peak[24] stores one value per hour; checking peak[hour] is O(1), as is the logical expression. The table adds 24 bool elements (commonly 24 bytes) but may make repeated checks simpler. Validate hour before indexing.

TASK 6 – Design the Control Structures (4 marks)

Pseudocode

```
BEGIN
  READ zone, hour, balance
  IF zone is invalid OR hour is outside 0..23 THEN
     REPORT invalid journey
     STOP this journey
  ENDIF
  SELECT base fare using zone
  peak <- (hour >= 8 AND hour < 10) OR (hour >= 17 AND hour < 20)
  IF peak THEN
     fare <- base + (base * 20 DIV 100)
  ELSE
     fare <- base
  ENDIF
  IF balance - fare >= 5000 THEN
     balance <- balance - fare
     PRINT fare and balance
  ELSE
     PRINT denied; do not change balance
  ENDIF
END
```

Use if–else for range conditions and eligibility, switch when mapping a small fixed enum to base fare, and ?: for short value selection only. A counted for loop suits a known number of journeys; a range-based for loop suits an existing collection; while is useful when input continues until a sentinel. continue skips an invalid journey; break exits a loop when processing should stop. A switch default case should reject unknown enum values.

Completed TODO program:

```
#include <iomanip>
#include <iostream>
using namespace std;
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak) {
   long long base = 0;
```

```
    switch (z) {
        case Zone::Inner: base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer: base = BASE_PAISE[2]; break;
        default: return -1;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}
bool canTravel(long long balance, long long f) {
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
int main() {
    long long balance = 10000;
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            cout << "Fare Rs " << f / 100 << '.' << setw(2)
                 << setfill('0') << f % 100 << setfill(' ');
        } else {
            cout << "Denied (low balance)";
        }
        cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

Expected output from the assignment and observed output:

```
Expected:
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54

Observed:
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Boundary hours: the correct predicate excludes 10:00 and 20:00. A fare array is also O(1) to index and stores three fare values; a fourth zone requires adding a new enum value and fare entry. A switch gives compiler-visible cases but must also be updated.

TASK 7 – From Procedural to Object-Oriented (4 marks)

Procedural style uses a balance variable passed to functions. OOP style keeps balance private and exposes canTravel() and deduct(). The class prevents negative or too-low balances because deduct() changes the balance only after canTravel() succeeds. If minimum balance changes, centralising it in MIN_BALANCE_PAISE means one constant is changed.

```
#include <iostream>
using namespace std;
constexpr long long MIN_BALANCE_PAISE = 5000;
bool proceduralCanTravel(long long balance, long long fare) {
    return fare >= 0 && balance - fare >= MIN_BALANCE_PAISE;
}
class SmartCard {
private:
    long long balancePaise;
public:
    explicit SmartCard(long long b) : balancePaise(b) {}
    bool canTravel(long long fare) const {
        return fare >= 0 && balancePaise - fare >= MIN_BALANCE_PAISE;
    }
    bool deduct(long long fare) {
        if (!canTravel(fare)) return false;
        balancePaise -= fare;
        return true;
    }
    long long balance() const { return balancePaise; }
};
int main() {
    long long balance = 10000, fare = 3000;
    cout << "Procedural allowed: " << boolalpha
         << proceduralCanTravel(balance, fare) << '\n';
    SmartCard card(10000);
    cout << "OOP allowed: " << card.canTravel(fare) << '\n';
    card.deduct(fare);
    cout << "OOP balance: " << card.balance() << " paise\n";
}
```

Observed output:

```
Procedural allowed: true
OOP allowed: true
OOP balance: 7000 paise
```

Alternative: a public balance data member does not change the O(1) time or O(1) storage per card, but any code could bypass validation and set the balance negative or below Rs 50. Private data and validated methods enforce the invariant structurally.

TASK 8 – Read and Print Data Safely (4 marks)

cin >> hour is concise, but invalid text sets the stream fail state. Check the extraction result, call cin.clear(), and discard the rest of the line with cin.ignore(numeric_limits<streamsize>::max(), '\n'). Validate the numeric range after parsing. <iomanip> provides setw and setfill for aligned money; fixed and setprecision(2) are useful for displaying rupees, though the stored amount should remain integer paise.

```
#include <iomanip>
#include <iostream>
#include <limits>
using namespace std;
int main() {
   int hour;
   cout << "Enter hour (0-23): ";
   if (!(cin >> hour)) {
      cout << "Invalid input: hour must be a number\n";
      cin.clear();
      cin.ignore(numeric_limits<streamsize>::max(), '\n');
      return 1;
   }
   if (hour < 0 || hour > 23) {
      cout << "Invalid input: hour must be 0-23\n";
      return 1;
   }
   cout << "Hour accepted: " << setw(2) << setfill('0') << hour << '\n';
}
```

Observed valid-input run (hour 12):

```
Enter hour (0-23): Hour accepted: 12
```

Observed invalid-input behaviour: with input 'abc', the program prints 'Invalid input: hour must be a number' and exits safely. It clears/discards bad input; in a repeated-input loop it could then prompt again.

Alternative: getline reads a whole line and stoi converts it inside try/catch. Both are O(L) in the line length L; getline needs a string buffer proportional to L. getline/stoi offers precise line-based recovery and can reject trailing characters with an index check; cin extraction is simpler for token-based input but needs stream-state recovery.

TASK 9 – Implement and Test the Fare Logic (4 marks)

The Task09 program implements zone validation, peak boundaries and a minimum-balance check.
Test table:

Case	Input	Expected (paise)	Actual (paise)	Result
Inner off-peak	Inner, 12	1000	1000	PASS
Middle at 08:00	Middle, 8	2400	2400	PASS
Outer at 17:00	Outer, 17	3600	3600	PASS
Outer at 10:00	Outer, 10	3000	3000	PASS
Outer at 20:00	Outer, 20	3000	3000	PASS
Unknown zone	Invalid, 12	-1	-1	PASS
Balance leaves exactly Rs 50	6000 balance, 1000 fare	ALLOW	ALLOW	PASS
Balance leaves Rs 49.99	5999 balance, 1000 fare	DENY	DENY	PASS

```
#include <iostream>
#include <string>
using namespace std;
enum class Zone { Inner, Middle, Outer, Invalid };
constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_PERCENT = 20;
long long baseFare(Zone z) {
   switch (z) {
      case Zone::Inner: return 1000;
      case Zone::Middle: return 2000;
      case Zone::Outer: return 3000;
      default: return -1;
   }
}
bool isPeak(int hour) {
   return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}
long long calculateFare(Zone z, int hour) {
   long long b = baseFare(z);
   if (b < 0 || hour < 0 || hour > 23) return -1;
   if (isPeak(hour)) b += b * PEAK_PERCENT / 100;
   return b;
}
bool canTravel(long long balance, long long fare) {
   return fare >= 0 && balance - fare >= MIN_BALANCE_PAISE;
}
int main() {
```

```
    struct Test { const char* name; Zone z; int hour; long long expected; };
    Test tests[] = {
        {"Inner off-peak", Zone::Inner, 12, 1000},
        {"Middle peak 8 AM", Zone::Middle, 8, 2400},
        {"Outer peak 17 PM", Zone::Outer, 17, 3600},
        {"Outer at 10 AM", Zone::Outer, 10, 3000},
        {"Outer at 8 PM", Zone::Outer, 20, 3000},
        {"Invalid zone", Zone::Invalid, 12, -1}
    };
    int passed = 0;
    for (const auto& t : tests) {
        long long actual = calculateFare(t.z, t.hour);
        bool ok = actual == t.expected;
        cout << t.name << ": expected=" << t.expected
             << ", actual=" << actual << " => " << (ok ? "PASS" : "FAIL") << '\n';
        if (ok) ++passed;
    }
    cout << "Passed " << passed << "/" << (sizeof(tests)/sizeof(tests[0])) << '\n';
    cout << "Balance 6000 paise, fare 1000: "
         << (canTravel(6000, 1000) ? "ALLOW" : "DENY") << '\n';
    cout << "Balance 5999 paise, fare 1000: "
         << (canTravel(5999, 1000) ? "ALLOW" : "DENY") << '\n';
}
```

Observed test output:

```
Inner off-peak: expected=1000, actual=1000 => PASS
Middle peak 8 AM: expected=2400, actual=2400 => PASS
Outer peak 17 PM: expected=3600, actual=3600 => PASS
Outer at 10 AM: expected=3000, actual=3000 => PASS
Outer at 8 PM: expected=3000, actual=3000 => PASS
Invalid zone: expected=-1, actual=-1 => PASS
Passed 6/6
Balance 6000 paise, fare 1000: ALLOW
Balance 5999 paise, fare 1000: DENY
```

Manual five-journey example (assuming all five are accepted and balance is sufficient): Inner off-peak Rs 10 + Middle peak Rs 24 + Outer off-peak Rs 30 + Inner peak Rs 12 + Middle off-peak Rs 20 = Rs 96. Peak boundaries must be tested explicitly: 08:00 and 17:00 apply surcharge; 10:00 and 20:00 do not. Negative recharge should be rejected before updating balance; non-numeric hour input must be detected via cin.fail().

Alternative: store test inputs and expected fares in a table and loop over them. For t tests the runtime is O(t), with O(t) storage for the table; adding a boundary case is easy and repeatable.

TASK 10 – Compare Expectations with Execution (4 marks)

For the comparison, assume every journey costs Rs 30.00. The expected totals are Rs 600,000 for 20,000 journeys; Rs 15,000,000 for 500,000 journeys; and Rs 90,000,000 for 3,000,000 journeys. Integer paise is exact. A double can represent these whole-rupee totals exactly at these magnitudes, though general decimal fractions may not be exact.

```
#include <climits>
#include <iomanip>
#include <iostream>
using namespace std;
int main() {
    const long long journeys[] = {20000, 500000, 3000000};
    const long long farePaise = 3000; // Rs 30.00
    cout << fixed << setprecision(2);
    for (long long n : journeys) {
        long long exactPaise = n * farePaise;
        double doubleRupees = static_cast<double>(n) * 30.0;
        cout << n << " journeys: integer-paise Rs "
             << static_cast<double>(exactPaise) / 100.0
             << ", double-rupee Rs " << doubleRupees << '\n';
    }
    long long networkPaise = 3000000LL * 3000LL;
    cout << "Network paise total=" << networkPaise << '\n';
    cout << "INT_MAX=" << INT_MAX << '\n';
    cout << "int can hold paise total: " << boolalpha
         << (networkPaise <= INT_MAX) << '\n';
}
```

Observed output:

```
20000 journeys: integer-paise Rs 600000.00, double-rupee Rs 600000.00
500000 journeys: integer-paise Rs 15000000.00, double-rupee Rs 15000000.00
3000000 journeys: integer-paise Rs 90000000.00, double-rupee Rs 90000000.00
Network paise total=9000000000
INT_MAX=2147483647
int can hold paise total: false
```

For 3,000,000 journeys, the total is 9,000,000,000 paise, which exceeds a typical 32-bit signed int maximum of 2,147,483,647. An int paise accumulator can overflow; long long can hold it. A double has broad range and is fast but does not guarantee exact decimal arithmetic. Each addition is O(1); int commonly uses 4 bytes, while long long and double commonly use 8 bytes.

Compiler flags -Wall -Wextra request common warnings. A signed/unsigned comparison may produce -Wsign-compare; unused variables may produce -Wunused-variable. The exact warnings depend on the program and compiler version. Check each task's compile output rather than claiming a warning that did not occur.

TASK 11 – Recommend and Defend the Final Design (4 marks)

Recommendation: retain the OOP structure, modify the fare calculation to use a single constexpr fare table and keep SmartCard balance private. Store money as long long paise, use enum class for zones, validate time and zone input, and centralise surcharge and minimum-balance constants. Test boundary times and the exact Rs 50 remaining-balance case before deployment.

- Fare revision: update one fare table and rerun the automated test suite.
- Limitations: current requirements do not specify the student/senior concession percentages or whether concession is applied before/after surcharge; clarify these before release.
- Network scaling: each journey is O(1); processing n journeys is O(n). A single card's state uses O(1) space, while a stored test table uses O(t).
- For high-volume production use, add configuration validation, audit logs, concurrency-safe account updates, and comprehensive integration tests.

```
#include <iostream>
using namespace std;
enum class Zone { Inner, Middle, Outer };
constexpr long long FARE_PAISE[] = {1000, 2000, 3000};
constexpr long long MIN_BALANCE_PAISE = 5000;
class SmartCard {
   long long balancePaise;
public:
   explicit SmartCard(long long initial) : balancePaise(initial) {}
   bool canTravel(long long fare) const {
      return fare >= 0 && balancePaise - fare >= MIN_BALANCE_PAISE;
   }
   bool deduct(long long fare) {
      if (!canTravel(fare)) return false;
      balancePaise -= fare;
      return true;
   }
   long long balance() const { return balancePaise; }
};
int main() {
   SmartCard card(10000);
   long long fare = FARE_PAISE[static_cast<int>(Zone::Middle)];
   cout << "Fare=" << fare << " paise\n";
   cout << "Travel=" << boolalpha << card.deduct(fare) << '\n';
   cout << "Balance=" << card.balance() << " paise\n";
}
```

Observed output:

```
Fare=2000 paise
Travel=true
Balance=8000 paise
```

Alternative: loading z fares from a configuration file adds O(z) startup time and O(z) memory for the table, but fares can be revised without recompiling. Validate file contents, missing entries and negative fares. Compiled constants are simpler and type-checked at build time; configuration is more flexible operationally.

TASK 12 – Re-evaluate and Adapt the Design (8 marks)

A. Impact analysis (2 marks)

- Add Zone::Airport and Rs 60 = 6000 paise to the fare table.
- Peak expression remains the same for weekday journeys; weekend discount is a separate policy.
- Add DayType and a MonthlyPass class that records two selected zones.
- Fare calculation and eligibility flow must distinguish pass-covered journeys from paid journeys.

B. Evaluate original design (2 marks)

A switch can absorb Airport by adding another case, but every switch over Zone must be updated. A fare table is easier to extend with a fourth entry, although enum and table order must remain consistent. SmartCard still protects balance, but a monthly pass is a different entitlement and should not be represented as an ordinary balance deduction.

Worked examples under the revised interpretation: Airport weekday off-peak = Rs 60; Airport weekday peak = Rs 72; weekend discount is 10% and does not combine with peak surcharge, so Airport weekend fare = Rs 54 even if the hour falls within the weekday peak window. This report assumes the weekend discount applies to the base fare and replaces, rather than combines with, the peak surcharge.

C. Adaptation / redesigned code (2 marks)

The following implementation uses a fare array, a separate DayType policy, and MonthlyPass coverage for two zones.

```
#include <array>
#include <iostream>
#include <string>
using namespace std;
enum class Zone { Inner, Middle, Outer, Airport };
constexpr array<long long, 4> FARE_PAISE = {1000, 2000, 3000, 6000};
constexpr long long MIN_BALANCE_PAISE = 5000;
enum class DayType { Weekday, Weekend };
long long calculateFare(Zone zone, int hour, DayType day) {
   long long fare = FARE_PAISE.at(static_cast<size_t>(zone));
   bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
   if (day == DayType::Weekend) return fare * 90 / 100; // no peak surcharge on weekends
```

```
    if (peak) fare += fare * 20 / 100;
    return fare;
}
class MonthlyPass {
    bool active;
    bool zones[4];
public:
    MonthlyPass(Zone first, Zone second) : active(true), zones{false,false,false,false} {
        zones[static_cast<int>(first)] = true;
        zones[static_cast<int>(second)] = true;
    }
    bool covers(Zone z) const { return active && zones[static_cast<int>(z)]; }
};
int main() {
    cout << "Airport weekday off-peak fare: "
         << calculateFare(Zone::Airport, 12, DayType::Weekday) << " paise\n";
    cout << "Airport weekday peak fare: "
         << calculateFare(Zone::Airport, 8, DayType::Weekday) << " paise\n";
    cout << "Airport weekend fare: "
         << calculateFare(Zone::Airport, 8, DayType::Weekend) << " paise\n";
    MonthlyPass pass(Zone::Inner, Zone::Airport);
    cout << boolalpha << "Pass covers Inner: " << pass.covers(Zone::Inner) << '\n';
    cout << "Pass covers Outer: " << pass.covers(Zone::Outer) << '\n';
}
```

Observed output:

```
Airport weekday off-peak fare: 6000 paise
Airport weekday peak fare: 7200 paise
Airport weekend fare: 5400 paise
Pass covers Inner: true
Pass covers Outer: false
```

D. Evidence-based defence (2 marks)

The tested program returns 6000 paise for Airport weekday off-peak, 7200 paise for Airport weekday peak, and 5400 paise for Airport weekend. The pass example covers Inner and Airport but not Outer. A table reduces repeated fare logic, and a dedicated pass object separates subscription entitlement from stored-value balance. Before deployment, add input checks for hour and zone, duplicate-zone policy for passes, pass expiry, and unit tests for all 4 zones and 24 hours.

Complexity: fare lookup and pass coverage are O(1); storing four fares and four zone flags is O(1) for this fixed network. For z zones, fare table/pass membership storage scales as O(z). The choice is MODIFY, not full redesign: preserve SmartCard encapsulation while separating fare policy and pass entitlements.

GitHub repository and execution checklist

- Create repository exactly as DSA01-<RegisterNumber>-Assignment1.
- Create Task01 through Task12 folders. Each folder should contain main.cpp, output.txt, and genuine output screenshots captured on your computer.
- Compile each program with: g++ -std=c++17 -Wall -Wextra main.cpp -o program.exe
- Run on Windows PowerShell with: .\program.exe (or ./program.exe in a compatible shell).
- For every task, commit progressively, for example: git add Task01 && git commit -m "Complete Task 1 analysis and demo". Repeat per task.
- Add README.md with name, register number, compiler/version, exact compile command, run instructions, and short task summaries.
- Push the repository and paste the working URL on the cover page and in Google Classroom.
- Review every code section and practise explaining it for the viva; do not submit code you cannot explain.

README.md starter

```
# DSA01-<RegisterNumber>-Assignment1
Name: <Your Name>
Register Number: <Your Register Number>
Compiler: g++ <version>
Compile: g++ -std=c++17 -Wall -Wextra main.cpp -o program.exe
Run (PowerShell): .\program.exe

## Tasks
- Task01: OOP objects and SmartCard invariant
- Task02: tokens, constants and enum class
- Task03: data types, limits and money precision
- Task04: implicit conversions and output prediction
- Task05: precedence and operators
- Task06: control structures and completed fare function
- Task07: procedural vs OOP SmartCard
- Task08: safe input and output
- Task09: fare validation and test cases
- Task10: totals and overflow comparison
- Task11: final design recommendation
- Task12: Airport zone, weekend discount and monthly pass
```

Execution verification summary

Folder	Status	Output file
Task01	COMPILED_AND_RAN	Task01/output.txt
Task02	COMPILED_AND_RAN	Task02/output.txt
Task03	COMPILED_AND_RAN	Task03/output.txt

Task04	COMPILED_AND_RAN	Task04/output.txt
Task05	COMPILED_AND_RAN	Task05/output.txt
Task06	COMPILED_AND_RAN	Task06/output.txt
Task07	COMPILED_AND_RAN	Task07/output.txt
Task08	RUN_EXIT_1	Task08/output.txt
Task09	COMPILED_AND_RAN	Task09/output.txt
Task10	COMPILED_AND_RAN	Task10/output.txt
Task11	COMPILED_AND_RAN	Task11/output.txt
Task12	COMPILED_AND_RAN	Task12/output.txt

Note: output.txt files record the local compile command, status and observed standard output. Capture terminal screenshots after you run the same programs on your own machine; screenshots are intentionally not fabricated in this report.